



Test Plan

Piano di test e suite `pytest` (`tests_agent.py`). Complementare alla [Technical Documentation](#).

1. Obiettivo e strategia

Verificare che l'agente sia **corretto** (trova le vulnerabilità reali, non ne inventa), **robusto** (regge modelli deboli, spegnimenti, input vari) e **deterministico** dove conta (verifica CodeQL, attribuzione CWE). La logica deterministica è testata senza LLM; l'LLM è coperto solo a livello end-to-end (opt-in).

2. Livelli di test

Livello	Marker pytest	Dipendenze	Velocità	Scopo
Unit	(nessuno)	nessuna	⚡	funzioni pure: parsing, rendering, normalizzazione, naming
Robustezza	(nessuno)	nessuna	⚡	fallback tool-call, fix db_path, checkpoint, zip
KB	(nessuno)	nessuna	⚡	integrità <code>cwe_wiki.json</code> , azioni, routing tool
Template	<code>codeql</code>	CodeQL	🐢	i template renderizzati compilano
Integrazione	<code>codeql</code>	CodeQL	🐢	finding reali su repo-fixture + caso negativo

Livello	Marker pytest	Dipendenze	Velocità	Scopo
End-to-end	llm	Ollama + CodeQL		agente completo → report con i CWE attesi

3. Come eseguire

```
# Solo veloci (unit + robustezza + KB) – default per lo svilu
ppo
python -m pytest tests_agent.py -m "not codeql and not llm"

# Con integrazione CodeQL (no LLM)
python -m pytest tests_agent.py -m "not llm"

# Tutto, incluso end-to-end col modello
python -m pytest tests_agent.py
```

4. Casi di test

4.1 Unit — recupero tool-call (fallback)

- call in blocco `json fenced` → recuperata correttamente
- report con un blocco json non-tool → NON scambiato per call
- nome tool sconosciuto → ignorato (no esecuzione arbitraria)
- `_balanced_objects` gestisce graffe annidate

4.2 Unit — sicurezza dell'iniezione

- `_render_names` quota gli identificatori; lista vuota → sentinella
- **nome malevolo** (es. `"]; evil //`) → scartato (no QL injection)
- `_render_consts` minuscolizza

4.3 Unit — CWE

- `_normalize_cwe` accetta `CWE-89` / `cwe-89` / `89`

- `_cwe_from_tags` estrae il CWE dai tag SARIF

4.4 Unit — SARIF / report

- `_parse_sarif` + `_extract_flow`: estrae cwe, sink, source, flow_steps
- `_default_report_path`: cartella per repo + slug del modello
- `_metadata_header`: contiene model/steps/cwes/findings

4.5 Robustezza

- `_resolve_db_path`: tiene un dir valido; sostituisce un placeholder col path reale
- checkpoint: save → reload roundtrip
- `_resolve_source`: estrae uno `.zip` e ritorna la cartella

4.6 Knowledge base

- schema di `cwe_wiki.json` valido (detection $\in$ {taint, api_misuse}; nomi presenti)
- `build_actions` produce il tool coerente col detection
- `cwe_knowledge` ritorna i due layer; `list_cwes` instrada al tool giusto

4.7 Template (CodeQL)

- il template taint renderizzato **compila** (`codeql query compile`)
- il template api_misuse renderizzato **compila**

4.8 Integrazione (CodeQL)

- su repo-fixture: `run_taint_query(["queryDB"], CWE-89)` → ≥ 1 finding in `app.py`
- `run_api_misuse_query(["md5"], ["md5", "sha1"], CWE-328)` → ≥ 2 finding
- **caso negativo**: repo pulita → 0 finding (no falsi positivi)

4.9 End-to-end (LLM)

- agente completo sulla repo-fixture → il report contiene `CWE-89` e `CWE-328`

5. Criteri di accettazione

Criterio	Soglia
Unit + robustezza + KB	100% pass
Template	entrambi compilano (rc=0)
Integrazione — recall fixture	SQLi trovata + ≥ 2 weak hash
Integrazione — falsi positivi	0 su repo pulita
Iniezione	nomi malevoli sempre scartati
End-to-end	report con i CWE attesi

6. Stato attuale

- **Unit + robustezza + KB: 26/26 pass** (~2s).
- Template + integrazione CodeQL: in validazione.
- End-to-end: opt-in (lento, dipende dal modello locale).

7. Estensioni future

- Repo-fixture multi-CWE con ground truth per misurare precision/recall per classe.
- CI: girare il livello veloce a ogni commit; CodeQL on-demand.
- Mock della MITRE API per testare `fetch_mitre` offline.